

Master Interview Q&A Bank – 80 LPA → 1 CR / Product Companies

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 17 Aug 2026 – 14 Feb 2027 (Days 1–182)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm**, **3:00–5:30pm**, **6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import <calendar/mern-mastery-study.ics>

140+ questions with full detailed answers for JavaScript, TypeScript, React, Node.js, Databases, System Design, LLD, Distributed Systems, Behavioral, and Salary Negotiation.

This bank is verbal Q&A only. Product companies also ask DSA, product HLD, backend MC, **frontend MC**, CS fundamentals, hiring-manager, and take-home — those types are listed and drilled in [20-1cr-product-company-complete.pdf](#) §2.

⚠ **STUDY TIERS – Do NOT read all at once**

Follow [00-LEARNING-PATH-COMPLETE.pdf](#) tier order. Reading Tier 4 before learning theory causes confusion.

Tier	Study when	Sections below
Tier 1	Month 1 (Days 1–28)	§1 JavaScript, §2 TypeScript
Tier 2	Month 2–3 (Days 29–84)	§3 React, §4 Node, §5 Databases
Tier 3	Month 4–5 (Days 85–140)	§7 LLD
Tier 4	Month 6 (Days 141–168)	§6 HLD, §8 Distributed, §9 Behavioral
Tier 5	Offer stage	§10 Salary

Daily: 2 Q&A from current tier only → explain aloud without looking.

Prerequisites before Tier 4: PDFs 10b → 10c → 10 → 11 (in that order).

Table of Contents

1. [JavaScript Deep \(15 Q&A\)](#)
2. [TypeScript \(10 Q&A\)](#)
3. [React / Frontend \(15 Q&A\)](#)
4. [Node.js / Backend \(15 Q&A\)](#)
5. [Databases \(15 Q&A\)](#)
6. [System Design HLD \(20 Q&A\)](#)
7. [LLD \(15 Q&A\)](#)
8. [Distributed Systems \(15 Q&A\)](#)
9. [Behavioral / Senior \(15 Q&A\)](#)
10. [Salary / Negotiation \(5 Q&A\)](#)

1. JavaScript Deep

TIER 1 — Study Month 1 (Days 1–28) • Prerequisites: [01-javascript-complete.pdf](#) §14–16

Q1: Explain the JavaScript event loop. How do microtasks and macrotasks differ?

Answer: The event loop is JavaScript's concurrency model for single-threaded execution. Call stack runs synchronous code; when empty, the loop processes the microtask queue (Promises, `queueMicrotask`, `MutationObserver`) completely before taking one macrotask (`setTimeout`, `setInterval`, I/O callbacks, UI events). This ordering explains why `Promise.then` runs before `setTimeout(0)`. In Node.js, the loop has phases: timers → pending callbacks → poll → check → close callbacks, with `process.nextTick` running between phases (higher priority than Promises). Understanding this prevents race bugs in async code and is essential for debugging "why did this log first?" interview questions.

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
// Output: 1, 4, 3, 2
```

Q2: What is a closure? Give a practical use case and a common pitfall.

Answer: A closure is a function that retains access to variables from its lexical scope even after the outer function returns. Practically, closures power private state, memoization, and factory functions. The classic pitfall is capturing loop variables in `var`-based loops — all callbacks share the same binding. Fix with `let` (block scope per iteration) or an IIFE. Closures can cause memory leaks if large objects are retained unnecessarily in long-lived callbacks (e.g., event handlers on removed DOM nodes).

```
function createCounter() {
  let count = 0; // private via closure
  return {
    increment: () => ++count,
    getCount: () => count,
  };
}
```

Q3: Explain prototypal inheritance vs classical inheritance.

Answer: JavaScript uses prototypal inheritance: objects delegate to their prototype via `[[Prototype]]` (accessed via `Object.getPrototypeOf` or `__proto__`). When a property isn't found on the object, the engine walks the prototype chain until `null`. ES6 `class` is syntactic sugar over constructor functions and `prototype`. `class Dog extends Animal` sets `Dog.prototype`'s prototype to `Animal.prototype`. Unlike classical inheritance, you can modify prototypes at runtime. For interviews, mention `Object.create(proto)` for explicit delegation without constructors.

```
const animal = { speak() { return 'sound'; } };  
const dog = Object.create(animal);  
dog.speak = () => 'woof'; // shadowing
```

Q4: What is the difference between `==` and `===`? When does coercion happen?

Answer: `===` is strict equality — no type coercion; compares type and value. `==` performs Abstract Equality Comparison: null/undefined equal each other, numbers and strings coerce, objects compared by reference after coercion. Examples: `0 == false` is true, `'' == false` is true, `null == undefined` is true, but `null === undefined` is false. Best practice: always use `===` except for null checks (`value == null` catches both null and undefined intentionally). Interviewers want you to know coercion rules, not use `==` in production.

Q5: Explain `this` binding rules in JavaScript.

Answer: `this` is determined by call site, not definition. Four rules in priority: (1) `new` binding — `this` is the new object; (2) explicit binding — `call` / `apply` / `bind`; (3) implicit binding — `obj.method()` sets `this` to `obj`; (4) default — `this` is `undefined` in strict mode, `globalThis` otherwise. Arrow functions don't have their own `this` — they inherit lexically. Common bug: extracting a method loses implicit binding; fix with `bind` or arrow function wrapper.

```
const obj = { name: 'A', getName() { return this.name; } };  
const fn = obj.getName;  
fn(); // undefined (strict) – lost binding
```

Q6: How does JavaScript memory management and garbage collection work?

Answer: JavaScript uses automatic garbage collection, primarily mark-and-sweep: GC marks reachable objects from roots (global, call stack, closures) and sweeps unreachable ones. V8 uses generational GC — young generation (Scavenge, short-lived objects) and old generation (Mark-Compact, long-lived). Memory leaks in JS are usually retained references: forgotten event listeners, global variables, closures holding large objects, detached DOM nodes, and unbounded caches. WeakMap/WeakSet allow garbage collection of keys despite being referenced. Use heap snapshots in Chrome DevTools to diagnose leaks.

Q7: What are Promises and how do async/await relate?

Answer: A Promise represents a future value with three states: pending, fulfilled, rejected. `.then / .catch` register callbacks in the microtask queue. `async/await` is syntactic sugar: `async` functions always return Promises; `await` pauses execution until Promise settles, without blocking the thread. Error handling: `try/catch` around `await`, or `.catch` on the returned Promise. Sequential vs parallel: `await a(); await b()` is sequential; `Promise.all([a(), b()])` is parallel. Never use `async` without `await` unless returning a Promise intentionally.

Q8: Explain debouncing vs throttling with use cases.

Answer: Debouncing delays execution until activity stops — ideal for search input (wait 300ms after last keystroke before API call). Throttling limits execution to once per interval — ideal for scroll/resize handlers (fire at most every 100ms). Debounce resets the timer on each trigger; throttle executes on the leading or trailing edge of the interval regardless of frequency. Both reduce expensive operations but serve different UX patterns. In React, implement with `useCallback` + refs or libraries like lodash.

```
function debounce(fn, delay) {  
  let timer;  
  return (...args) => {  
    clearTimeout(timer);  
    timer = setTimeout(() => fn(...args), delay);  
  };  
}
```

Q9: What is the Temporal Dead Zone (TDZ)?

Answer: TDZ is the period between entering a scope and the `let` / `const` declaration being initialized. Accessing the variable during TDZ throws `ReferenceError`, unlike `var` which hoists with `undefined`. `typeof undeclaredVar` returns `'undefined'`, but `typeof letVar` in TDZ throws. TDZ prevents using block-scoped variables before declaration, catching bugs from hoisting confusion. `const` must be initialized at declaration; `let` can be declared without assignment but remains in TDZ until the declaration line executes.

Q10: Explain event delegation and why it's useful.

Answer: Event delegation attaches one listener to a parent instead of many on children, relying on event bubbling. Click on `<li>` bubbles to `<ul>` where a single handler checks `event.target`. Benefits: works for dynamically added elements, fewer listeners (memory/performance), simpler cleanup. Caveats: not all events bubble (focus/blur use focusin/focusout); need to filter targets carefully. In React, synthetic event delegation to root is built-in, but the concept applies to vanilla JS and performance optimization discussions.

Q11: What are WeakMap and WeakSet? When would you use them?

Answer: WeakMap keys must be objects; keys are weakly held — if no other references exist, GC collects the key and entry automatically. WeakSet holds objects weakly with same GC behavior. Neither is iterable; no `size` property. Use cases: storing private/metadata about DOM nodes without preventing GC, caching computed results per object, tracking object

presence without memory leaks. Unlike Map, you can't use primitives as keys. Perfect for associating data with objects you don't own (e.g., third-party instances).

Q12: Explain currying and partial application.

Answer: Currying transforms `f(a, b, c)` into `f(a)(b)(c)` — each call takes one argument and returns a function until all args collected. Partial application fixes some arguments: `f(a, b, c) → g(b, c)` where `a` is preset. Currying enables function composition and configuration; partial application creates specialized functions from general ones. In interviews, show both: curry for reusable pipelines, partial for pre-configuring API calls with base URL/token.

```
const curry = (fn) => (...args) =>
  args.length >= fn.length ? fn(...args) : (...more) => curry(fn)(...args, ...more)
const add = (a, b, c) => a + b + c;
const add5 = curry(add)(2)(3); // returns function waiting for c
```

Q13: What is the module pattern and how do ES modules differ from CommonJS?

Answer: ES modules (`import` / `export`) are static — imports resolved at parse time, enabling tree-shaking and circular dependency analysis. CommonJS (`require` / `module.exports`) is dynamic — loads at runtime, synchronous, default in Node.js historically. ESM has live bindings for exports; CJS copies values at require time. Node.js now supports ESM with `"type": "module"` in package.json. Browser ESM uses `<script type="module">` with strict mode and deferred execution. For 80 LPA interviews, mention interoperability challenges and that bundlers abstract most differences.

Q14: How does `Object.freeze` differ from `Object.seal` and `Object.preventExtensions`?

Answer: `preventExtensions`: no new properties can be added; existing properties can be deleted/modified. `seal`: `preventExtensions` + existing properties cannot be deleted (but values

can change if writable). `freeze`: seal + existing property values cannot change (writable and configurable false). Shallow only — nested objects remain mutable unless recursively frozen. Use `Object.isFrozen(obj)` to check. For immutability in React/state management, prefer spread operators or Immer over deep freeze due to performance cost.

Q15: Explain `requestAnimationFrame` vs `setTimeout` for animations.

Answer: `requestAnimationFrame` (rAF) syncs callbacks to display refresh rate (~60fps), pauses in background tabs (saves battery), and batches DOM reads/writes optimally. `setTimeout` is timer-based with no sync to paint cycle — causes jank and drift. rAF receives a timestamp for smooth interpolation. Pattern: update state in rAF callback, never do heavy computation inside. For React animations, CSS transitions/Framer Motion abstract this, but understanding rAF shows performance awareness for custom canvas/WebGL work.

2. TypeScript

TIER 1 — Study Month 1 (Days 22–28) • Prerequisites: `02-typescript-complete.pdf`

Q1: What are the key benefits of TypeScript over JavaScript for large codebases?

Answer: TypeScript adds static typing, enabling compile-time error detection, IDE autocomplete, and safe refactoring across thousands of files. Interfaces and generics document API contracts without separate documentation. Strict mode (`strict: true`) catches null/undefined bugs via `strictNullChecks`. For 80 LPA teams, TypeScript reduces production bugs and onboarding time — new engineers understand function signatures from types alone. Trade-off: build step and occasional type gymnastics, but ROI is positive above ~10K LOC.

Q2: Explain `interface` vs `type` alias. When to use each?

Answer: Interfaces are extendable via `extends` and declaration merging (same name merges). Type aliases can represent unions, intersections, primitives, tuples — more flexible. Use interfaces for object shapes and public APIs (extendable by consumers). Use types for unions (`type Status = 'active' | 'inactive'`), mapped types, and conditional types. Both work for object shapes; consistency in a codebase matters more than the choice. Interview tip: mention declaration merging as the unique interface feature.

```
interface User { id: string; name: string; }  
interface User { email: string; } // merges  
type Result<T> = { success: true; data: T } | { success: false; error: string };
```

Q3: What are generics? Provide a real-world example.

Answer: Generics parameterize types, enabling reusable components that work with multiple types while maintaining type safety. `Array<T>` is generic — `Array<string>` ensures only strings. Real-world: API response wrapper `ApiResponse<T>` where `T` is the data shape per endpoint. Constraints with `extends` limit generic types: `<T extends Identifiable>` ensures `T` has `id`. Avoid `any` — generics preserve type information through transformations.

```
async function fetchApi<T>(url: string): Promise<ApiResponse<T>> {  
  const res = await fetch(url);  
  return res.json() as Promise<ApiResponse<T>>;  
}  
const user = await fetchApi<User>('/api/user/1');
```

Q4: Explain utility types: `Partial`, `Pick`, `Omit`, `Record`.

Answer: `Partial<T>` makes all properties optional — useful for update payloads. `Pick<T, Keys>` selects subset of properties. `Omit<T, Keys>` excludes properties. `Record<Keys, Type>` creates object type with specified keys and value type. Combined: `Partial<Pick<User, 'name'>`

| 'email'>> for partial user updates. These eliminate manual type duplication and stay in sync when base types change. Senior engineers use them in API layers and form handling daily.

Q5: What is type narrowing? How do discriminated unions work?

Answer: Type narrowing refines a broad type to a specific type via control flow analysis. Methods: `typeof`, `instanceof`, `in` operator, equality checks, and discriminated unions. Discriminated unions have a common literal property (discriminant) enabling exhaustive switch: `type Shape = { kind: 'circle'; radius: number } | { kind: 'rect'; w: number; h: number }`. TypeScript narrows based on `kind`. Use `never` in default case to ensure exhaustiveness — compiler error if new variant added without handling.

Q6: Explain `unknown` vs `any` vs `never`.

Answer: `any` disables type checking — escape hatch, avoid in production. `unknown` is the type-safe top type — must narrow before use (`typeof`, `instanceof`, type guards). `never` represents values that never occur — empty union, functions that always throw, exhaustive check default. Prefer `unknown` over `any` for external data (API responses, `JSON.parse`). `never` proves exhaustiveness in switch statements and intersection type elimination.

Q7: What are conditional types and infer?

Answer: Conditional types select types based on condition: `T extends U ? X : Y`. `infer` extracts types within conditional branches — powers advanced utility types. Example: `ReturnType<T>` extracts function return type using `infer R`. Used in library types (React's `ComponentProps<T>`). For interviews, explain the concept; you don't need to write complex conditional types live, but reading them in library code is expected at L5.

```
type ReturnType<T> = T extends (...args: any[]) => infer R ? R : never;
type ElementType<T> = T extends infer U[] ? U : never;
```

Q8: How do you handle third-party libraries without types?

Answer: Options in order: (1) Check `@types/package-name` on DefinitelyTyped; (2) Use built-in types if library ships them; (3) Write declaration file `types/package.d.ts` with `declare module 'package'`; (4) Last resort: `@ts-expect-error` with comment explaining why. For production, invest in proper declarations — untyped libraries are tech debt. Module augmentation extends existing types when library types are incomplete.

Q9: Explain strict mode flags that matter most.

Answer: `strict: true` enables all strict checks. Key individual flags: `strictNullChecks` — null/undefined are distinct types, prevents #1 TS benefit; `noImplicitAny` — error on untyped parameters; `strictFunctionTypes` — contravariant function parameter checking; `noUncheckedIndexedAccess` — index access returns `T | undefined`. Enable incrementally on existing codebases. For greenfield, always `strict: true`. Interview answer: `strictNullChecks` alone prevents more bugs than any other single flag.

Q10: What is module augmentation? Give an example.

Answer: Module augmentation adds types to existing modules without modifying source. Declare same module path and merge interfaces. Use case: extend Express `Request` with custom properties (user, session). Also works for global scope augmentation. Must be in a `.ts` file included in compilation. Pattern essential for middleware that attaches data to request objects in typed Node.js applications.

```
declare global {
  namespace Express {
    interface Request {
      user?: { id: string; role: string };
    }
  }
}
```

3. React / Frontend

TIER 2 — Study Month 2 (Days 29–56) • Prerequisites: `00b-html-css-web-fundamentals.pdf` , `03-react-complete.pdf`

Q1: Explain the React rendering lifecycle and reconciliation.

Answer: React renders by calling component functions, producing a Virtual DOM tree. Reconciliation compares new VDOM with previous (diffing), computing minimal DOM mutations. Keys help identify list items across renders — stable unique keys prevent incorrect reuse. Render phase is pure (no side effects); commit phase applies DOM updates and runs layout effects. React 18 concurrent features allow interruptible rendering — urgent updates (typing) can interrupt slow renders (search results). Understanding this explains why inline object/array props cause re-renders (new reference each render).

Q2: What causes unnecessary re-renders and how do you fix them?

Answer: Re-renders occur when state/props/context change in a component or ancestor. Unnecessary re-renders: passing new object/function references as props, context value changes affecting all consumers, parent re-render cascading to memoized-less children. Fixes: `React.memo` for pure components, `useMemo` / `useCallback` for stable references, splitting context (state vs dispatch), component composition (children as props pattern), virtualization for long lists. Profile with React DevTools Profiler before optimizing — premature memoization adds complexity without benefit.

Q3: Explain `useEffect` vs `useLayoutEffect` vs `useInsertionEffect`.

Answer: `useEffect` runs asynchronously after paint — ideal for data fetching, subscriptions, non-visual side effects. `useLayoutEffect` runs synchronously after DOM mutations but before paint — use for DOM measurements, preventing visual flicker. `useInsertionEffect` runs before DOM mutations — CSS-in-JS libraries inject styles here. Rule: default to `useEffect` ;

reach for `useLayoutEffect` only when you measure DOM or need synchronous update before user sees screen. Both run on client only — not during SSR.

Q4: What is SSR hydration and what causes hydration mismatches?

Answer: SSR renders HTML on server; client React "hydrates" by attaching event listeners and reconciling with server HTML. Hydration mismatch occurs when client render output differs from server HTML — React warns and may regenerate subtree. Causes: `Date.now()`, `Math.random()`, browser-only APIs during render, invalid HTML nesting (`<p>` inside `<p>`), locale/timezone differences, conditional rendering based on `window`. Fix: render consistent content on server and client initially; use `useEffect` for client-only content; `suppressHydrationWarning` only for known unavoidable diffs (timestamps).

Q5: Explain React Server Components (RSC) and their benefits.

Answer: RSC run exclusively on server — zero client JavaScript bundle for server components. They can directly access databases, file system, and secrets without API routes. Client Components (`'use client'`) handle interactivity. Composition: server components import and render client components (passing serializable props). Benefits: reduced bundle size, faster initial load, colocated data fetching. Trade-offs: no hooks/state in server components, serialization limits on props. Next.js App Router is the primary adoption path for RSC in production.

Q6: How does React Context work and when should you avoid it?

Answer: Context provides value to descendant tree without prop drilling. Provider re-render causes all consuming components to re-render when value changes (reference equality). Good for: theme, locale, auth user, feature flags — infrequent changes. Avoid for: frequently changing state (use state management library or component state), performance-critical lists, data that only 2-3 levels need (prop drilling may be simpler). Optimization: split contexts, memoize value object, use context selectors (external libraries) or Zustand/Jotai for granular subscriptions.

Q7: Explain code splitting and lazy loading in React.

Answer: Code splitting breaks bundle into chunks loaded on demand. `React.lazy(() => import('./Component'))` + `Suspense` boundary shows fallback while loading. Route-based splitting (React Router lazy routes, Next.js dynamic imports) is most effective — users download only current page code. Prefetch on hover/link visibility for perceived performance. Measure with webpack-bundle-analyzer. For 80 LPA: mention SSR implications — lazy components need Suspense boundaries; Next.js `dynamic()` handles SSR disable option for client-only components.

```
const Dashboard = React.lazy(() => import('./Dashboard'));
function App() {
  return (
    <Suspense fallback={<Spinner />}>
      <Dashboard />
    </Suspense>
  );
}
```

Q8: What are React 18 concurrent features?

Answer: Concurrent rendering allows React to interrupt, pause, and resume rendering work. Features: `startTransition` marks non-urgent updates (search filtering) so typing stays responsive; `useDeferredValue` defers updating a value; Suspense for data fetching (with compatible libraries); automatic batching groups all state updates regardless of source (setTimeout, promises). `createRoot` replaces `ReactDOM.render`. These features improve UX under load without manual debouncing for many scenarios.

Q9: How do you optimize Core Web Vitals (LCP, INP, CLS)?

Answer: LCP (Largest Contentful Paint): optimize hero image (preload, WebP/AVIF, proper sizing, CDN), reduce TTFB (SSR/edge caching), eliminate render-blocking resources. INP (Interaction to Next Paint): reduce JS main thread work, code split, web workers for heavy computation, optimize event handlers. CLS (Cumulative Layout Shift): set explicit width/height on images/embeds, reserve space for dynamic content, avoid inserting content above existing

content, use `font-display: swap` with metric-matched fallbacks. Measure with Lighthouse and real user monitoring (CrUX).

Q10: Explain state management choices: Context vs Redux vs Zustand.

Answer: Context: built-in, good for low-frequency global state, poor for high-frequency updates. Redux: predictable with actions/reducers, excellent DevTools, middleware ecosystem — overhead for simple apps. Redux Toolkit simplifies boilerplate significantly. Zustand/Jotai: minimal API, no providers needed, granular subscriptions, smaller bundle. Choose based on complexity: component state → Context → Zustand → Redux. For 80 LPA interviews, justify choice with team size, state complexity, and DevTools/debugging needs — no single correct answer.

Q11: What is the Virtual DOM and is it always faster than direct DOM?

Answer: Virtual DOM is an in-memory representation of UI — React diffs VDOM trees to compute minimal real DOM operations. It's not always faster than hand-tuned direct DOM manipulation for simple updates. Benefits: declarative programming, batched updates, cross-platform (React Native). Cost: diffing overhead and memory for VDOM tree. React's advantage is developer productivity and predictable performance at scale, not raw speed vs optimized vanilla JS. Concurrent mode further optimizes by prioritizing updates.

Q12: Explain controlled vs uncontrolled components.

Answer: Controlled: React state is single source of truth — `value={state}` + `onChange` updates state. Enables validation, formatting, conditional disabling on every keystroke. Uncontrolled: DOM holds state — use `ref` to read values on submit. Less code for simple forms. React Hook Form uses uncontrolled internally with refs for performance (fewer re-renders). For complex forms with validation, controlled or Hook Form; for simple login, either works. File inputs must be uncontrolled (or use File API).

Q13: How does error boundary work? What errors does it NOT catch?

Answer: Error boundaries are class components (or react-error-boundary library) with `static getDerivedStateFromError` and `componentDidCatch` — catch render errors in children, show fallback UI, log to monitoring. Do NOT catch: event handler errors (use try/catch), async errors (use `.catch`), SSR errors, errors in boundary itself. Pattern: wrap route-level components with boundaries for graceful degradation. In React 19+, expect improved error handling primitives.

Q14: Explain CSS-in-JS trade-offs vs Tailwind/CSS Modules.

Answer: CSS-in-JS (styled-components, Emotion): scoped styles, dynamic theming, co-location — runtime cost (unless zero-runtime like Linaria), larger bundles. CSS Modules: build-time scoping, no runtime, standard CSS syntax — less dynamic theming. Tailwind: utility-first, purge unused classes, consistent design system, fast iteration — JSX can get verbose, learning curve. For 80 LPA: discuss team preference and performance. Trend: Tailwind + CSS Modules for static, CSS-in-JS declining due to RSC compatibility concerns (styles must not depend on client runtime).

Q15: How do you implement infinite scroll performantly?

Answer: Use Intersection Observer on a sentinel element at list bottom — trigger fetch when visible. Virtualize the list (react-window/react-virtuoso) to render only visible items — critical for 1000+ items. Debounce scroll handlers if not using Intersection Observer. Cache fetched pages; deduplicate requests. Show skeleton loaders for perceived performance. For SSR, render first page server-side. Mention cursor-based pagination (not offset) for consistent results when data changes during scroll.

4. Node.js / Backend

TIER 2 — Study Month 3 (Days 57–84) • Prerequisites: `05-nodejs-express-complete.pdf`, `10b-networking-http-fundamentals.pdf`

Q1: Explain the Node.js event loop phases.

Answer: Node.js event loop has 6 phases: (1) timers — execute `setTimeout` / `setInterval` callbacks; (2) pending callbacks — I/O callbacks deferred to next cycle; (3) idle, prepare — internal use; (4) poll — retrieve new I/O events, execute I/O callbacks; (5) check — `setImmediate` callbacks; (6) close callbacks — e.g., `socket.on('close')`. Between phases, Node processes `process.nextTick` queue (all) then microtask queue (Promises). This explains why `setImmediate` vs `setTimeout(0)` ordering differs between main module and I/O cycles.

Q2: How does Node.js handle concurrency with a single thread?

Answer: Node.js uses non-blocking I/O with event loop — single thread handles JS, but libuv thread pool (default 4 threads) handles file I/O, DNS, crypto. Network I/O is truly async via OS mechanisms (epoll/kqueue). CPU-bound tasks block the event loop — fix with worker threads (`worker_threads` module) or separate processes. Cluster module forks processes per CPU core for horizontal scaling on one machine. Never do heavy computation in request handlers without offloading.

Q3: Explain streams in Node.js. What are the four types?

Answer: Streams process data chunk-by-chunk without loading entire dataset into memory. Four types: Readable (source — `fs.createReadStream`), Writable (destination — `fs.createWriteStream`), Duplex (both — TCP socket), Transform (modify data — `zlib.createGzip`). Pipe connects streams: `readable.pipe(writable)`. Backpressure: when writable buffer full, pause readable until drained — `.pipe()` handles automatically. Use for large file processing, video streaming, log parsing — essential for memory-efficient backend at scale.

Q4: What is middleware in Express and how does the chain work?

Answer: Middleware functions have signature `(req, res, next)` — execute sequentially in order defined. Each can modify req/res, end response, or call `next()` to pass control. Order

matters: error handlers have 4 params `(err, req, res, next)` and must be defined last. Common stack: helmet (security) → cors → body parser → auth → routes → error handler. Async middleware must catch errors and pass to `next(err)` or use `express-async-errors` wrapper. Understanding chain is fundamental for debugging "why didn't my route handler run?"

Q5: How do you handle errors in async Express routes?

Answer: Async errors don't automatically reach Express error handlers — rejected Promises crash the process in older Node. Solutions: wrap in try/catch + `next(err)`, use `express-async-handler` wrapper, or Express 5 (native async support). Centralized error handler returns consistent JSON: `{ error: message, code: statusCode }`. Distinguish operational errors (expected — 404, validation) from programmer errors (log + 500). Never expose stack traces in production responses.

```
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);
```

Q6: Explain clustering vs worker threads in Node.js.

Answer: Cluster module forks separate Node processes sharing server port via OS scheduling — true parallelism for CPU-bound work across cores, fault isolation (one crash doesn't kill all). Worker threads share memory (SharedArrayBuffer) within a process — lower overhead than cluster, ideal for CPU-intensive tasks offloaded from main thread. Cluster for scaling HTTP servers; worker threads for parallel computation (image processing, JSON parsing). Both address single-thread limitation but at different granularity. PM2/K8s often replace manual cluster management in production.

Q7: What is the difference between `process.nextTick` and `setImmediate`?

Answer: `process.nextTick` schedules callback before event loop continues — runs after current operation, before any I/O phase. `setImmediate` runs in check phase, after poll phase I/O callbacks. In main module: `nextTick` always before `setImmediate`. In I/O callback:

setImmediate before nextTick (usually). Excessive nextTick calls starve I/O — "nextTick starvation." Rule: use nextTick for deferring until sync code completes (e.g., emit 'ready' after constructor); setImmediate for deferring to next event loop iteration.

Q8: How do you implement authentication in Node.js APIs?

Answer: Common patterns: JWT (stateless — sign payload with secret, verify on each request; short expiry + refresh tokens), session cookies (stateful — store session server-side in Redis, cookie contains session ID). Implementation: middleware extracts token/cookie, verifies, attaches user to req. Security: httpOnly + secure + sameSite cookies, bcrypt/argon2 for password hashing (12+ rounds), rate limit login endpoints, rotate refresh tokens, never store plaintext passwords. OAuth2 for third-party login. For 80 LPA, discuss trade-offs: JWT scalability vs session revocation ease.

Q9: Explain environment configuration best practices.

Answer: Use environment variables for config that varies per environment (DB URL, API keys, port). Never commit secrets — use `.env` locally (in `.gitignore`), vault/K8s secrets in production. Validate env at startup with Zod/joi — fail fast on missing vars. Schema: `DATABASE_URL` (required), `PORT` (default 3000), `NODE_ENV` (development/staging/production). 12-factor app principle: strict separation of config from code. Different configs for dev/staging/prod without code changes.

Q10: What is backpressure and how do you handle it?

Answer: Backpressure occurs when data producer is faster than consumer — buffers grow, memory exhausts. In Node streams, writable emits 'drain' when buffer empties after `write()` returns false. HTTP: slow client reading response — pause upstream until drain. Database: batch inserts instead of individual writes. Message queues: consumer lag monitoring, auto-scaling consumers. Pattern: always respect backpressure signals rather than unbounded buffering — critical for production stability at scale.

Q11: How do you structure a large Node.js application?

Answer: Common patterns: layered architecture — routes/controllers → services (business logic) → repositories (data access). Feature-based folders for larger apps: `features/users/`, `features/orders/` each with controller, service, repository. Dependency injection (manual or tsyringe) for testability. Shared: middleware, utils, config, types. Avoid: routes calling database directly, god service classes, circular dependencies. For microservices: each service is a separate deployable with own repo or monorepo with clear boundaries.

Q12: Explain caching strategies in Node.js backends.

Answer: Layers: in-memory (node-cache/lru-cache — fast, not shared across instances), Redis (shared, TTL, pub/sub invalidation), CDN (static assets, cacheable API responses). Patterns: cache-aside (app checks cache, on miss loads DB, writes cache), write-through (write to cache and DB simultaneously), TTL-based expiry. Cache invalidation: hardest problem — event-driven invalidation on writes, versioned keys. Never cache user-specific data with shared keys. Set appropriate TTLs — balance freshness vs hit rate.

Q13: What is graceful shutdown and why does it matter?

Answer: Graceful shutdown handles SIGTERM/SIGINT by: stop accepting new connections, finish in-flight requests (with timeout), close database/Redis connections, flush logs, then exit. Without it, deploys kill mid-request causing 502 errors and data inconsistency. Implementation: track active connections, `server.close()` stops new, set force-exit timeout (30s). K8s sends SIGTERM before SIGKILL — must complete within `terminationGracePeriodSeconds`. Essential for zero-downtime deployments.

```
process.on('SIGTERM', () => {
  server.close(() => { db.disconnect(); process.exit(0); });
  setTimeout(() => process.exit(1), 30000);
});
```

Q14: How do you debug memory leaks in Node.js?

Answer: Symptoms: increasing heap over time, eventual OOM crash, growing response times. Tools: `node --inspect` + Chrome DevTools heap snapshots (compare over time), clinic.js, `process.memoryUsage()`. Common causes: global arrays/maps growing unbounded, forgotten event listeners, closures retaining request context, cached objects without TTL. Fix: bounded caches, remove listeners on cleanup, weak references where appropriate, monitor heap in production with alerts on growth trend.

Q15: Explain API versioning strategies.

Answer: URL path (`/v1/users`) — simplest, most visible, easy routing. Header (`Accept: application/vnd.api.v2+json`) — clean URLs, harder to test in browser. Query param (`?version=2`) — easy but messy. Best practice: version public APIs, maintain backward compatibility for 12+ months, deprecation headers (`Sunset` , `Deprecation`). Internal APIs can break freely. Document breaking changes in changelog. Contract tests in CI prevent accidental breaking changes — essential for platforms with external integrators.

5. Databases

TIER 2 — Study Month 3 (Days 71–84) • Prerequisites: `06-databases-complete.pdf`

Q1: Explain B-tree indexes and when they help vs hurt.

Answer: B-tree (B+ tree in most databases) is the default index structure — balanced tree with $O(\log n)$ search, range scans, and ordered traversal. Helps: WHERE clauses on indexed columns, JOIN conditions, ORDER BY on indexed columns, MIN/MAX. Hurts: write overhead (index must update on INSERT/UPDATE/DELETE), storage space, wrong index choice causes full table scans anyway. Composite index column order matters — leftmost prefix rule. Index `(a, b, c)` helps queries on `a` , `a+b` , `a+b+c` but not `b` alone or `c` alone.

Q2: What is the N+1 query problem and how do you fix it?

Answer: N+1 occurs when fetching N parent records triggers N additional queries for related data — 1 query for list + N for each item's relations. Example: get 100 posts, then query author for each post individually. Fixes: eager loading/JJOIN (`SELECT * FROM posts JOIN users`), batch loading (`WHERE id IN (...)`), DataLoader pattern (batch + cache within request), ORM `include` / `populate` options. Detect with query logging — if you see repeated similar queries in loops, you have N+1. At scale, N+1 can mean thousands of DB round trips per request.

Q3: Explain ACID properties with examples.

Answer: Atomicity: transaction全部 succeeds or全部 rolls back — bank transfer debits and credits together. Consistency: transaction moves DB from one valid state to another — constraints enforced (FK, unique, check). Isolation: concurrent transactions don't interfere — levels: Read Uncommitted, Read Committed, Repeatable Read, Serializable. Durability: committed data survives crash — WAL/redo logs. Trade-off: higher isolation = less concurrency. PostgreSQL default Read Committed; MySQL InnoDB Repeatable Read. Choose isolation based on consistency needs vs performance.

Q4: What are database transactions and isolation levels?

Answer: Transaction groups operations into atomic unit. Isolation levels control visibility of concurrent changes: Read Uncommitted (dirty reads possible — rarely used), Read Committed (no dirty reads, non-repeatable reads possible — PostgreSQL default), Repeatable Read (consistent snapshot within transaction — MySQL default), Serializable (full isolation, may cause serialization failures). Phenomena: dirty read, non-repeatable read, phantom read. Use explicit transactions for multi-step operations: `BEGIN; ... COMMIT;` with appropriate isolation for financial operations.

Q5: Explain database sharding vs partitioning.

Answer: Partitioning divides tables within one database instance — range, list, hash partitions. Improves query performance via partition pruning, manageable on single server. Sharding distributes data across multiple database instances — horizontal scaling beyond single machine limits. Shard key choice is critical — must align with query patterns to avoid cross-shard joins. Sharding adds complexity: rebalancing, cross-shard transactions, global IDs. Start with read replicas and partitioning; shard when single instance limits hit (typically 1-10TB+ or write throughput ceiling).

Q6: What is a covering index?

Answer: Covering index includes all columns needed by a query — index-only scan without accessing table heap. Example: query `SELECT name, email FROM users WHERE status = 'active'` with index on `(status, name, email)` — all data from index. Benefits: eliminates random heap lookups, faster reads. Trade-off: wider index, more write overhead. Use `EXPLAIN ANALYZE` to verify index-only scans. Include frequently queried columns in composite indexes strategically.

Q7: Explain CAP theorem and its practical implications.

Answer: CAP: in network partition, choose Consistency (all nodes see same data) or Availability (every request gets response) — not both during partition. Partition tolerance is mandatory in distributed systems. CP systems: MongoDB (with strong consistency), etcd, ZooKeeper — may reject writes during partition. AP systems: Cassandra, DynamoDB (eventual consistency) — always accept writes, reconcile later. Most production systems are PA with configurable consistency — not pure CAP choice. Base (Basically Available, Soft state, Eventual consistency) is practical extension.

Q8: What is database connection pooling and why is it needed?

Answer: Connection pooling maintains reusable database connections instead of opening/closing per request. Creating connections is expensive (TCP handshake, auth, memory). Pool settings: min/max connections, idle timeout, connection lifetime. Too few connections: request queuing under load. Too many: database overwhelmed (PostgreSQL `max_connections`). Rule of thumb: pool size $\approx (\text{CPU cores} \times 2) + \text{spindle count}$ for traditional DBs; less for SSD/cloud. PgBouncer for PostgreSQL connection pooling at scale — transaction vs session pooling modes.

Q9: Explain optimistic vs pessimistic locking.

Answer: Pessimistic: lock row during read (`SELECT FOR UPDATE`) — prevents concurrent modifications, may cause deadlocks and reduced throughput. Optimistic: read without lock, check version on write (`UPDATE ... WHERE id = ? AND version = ?`) — if version changed, retry. Optimistic suits low-contention (most web apps); pessimistic suits high-contention (inventory, seat booking). Implement optimistic with version column or timestamp. BookMyShow seat booking often uses optimistic: update only if status still 'available'.

Q10: What are database migrations and best practices?

Answer: Migrations are versioned schema changes tracked in code — Flyway, Liquibase, Prisma Migrate, Knex. Best practices: never modify deployed migrations, backward-compatible changes (add column nullable before backfill), separate DDL from DML, test on production-like data volume, plan rollback strategy. Dangerous operations: adding NOT NULL without default, dropping columns in use, large table ALTER (use `pt-online-schema-change`). Blue-green deployments require migrations compatible with both old and new code versions.

Q11: Explain read replicas and replication lag.

Answer: Read replicas copy data from primary (master) asynchronously or semi-synchronously — scale read traffic. Lag: time between write on primary and visibility on replica — milliseconds

to seconds. Problems: read-your-writes violation (user creates post, refresh shows empty), stale data in dashboards. Solutions: read from primary after write (sticky sessions), version/timestamp checks, accept eventual consistency for non-critical reads. Monitor replication lag — alert if exceeds threshold. Promote replica to primary during failover.

Q12: What is a database deadlock and how do you prevent it?

Answer: Deadlock: two transactions hold locks each other needs — circular wait. Database detects and aborts one transaction (deadlock victim). Prevention: acquire locks in consistent order (always lock table A before B), keep transactions short, use appropriate isolation level, retry on deadlock error. Detection in PostgreSQL: `pg_locks` view. Application pattern: catch deadlock error code (40001 in PostgreSQL), exponential backoff retry. Design to minimize lock contention — smaller transactions, index to reduce lock scope.

Q13: Explain SQL query optimization process.

Answer: Process: (1) EXPLAIN ANALYZE the query — check scan type (seq scan vs index scan), rows estimated vs actual, cost; (2) identify missing indexes on WHERE/JOIN columns; (3) rewrite query — avoid SELECT *, subqueries vs JOINS, EXISTS vs IN; (4) update statistics (ANALYZE); (5) consider materialized views for expensive aggregations. Red flags: sequential scan on large tables, nested loop with high row counts, sort operations on large datasets. Index isn't free — verify it helps target query without hurting writes excessively.

Q14: What is eventual consistency and where is it acceptable?

Answer: Eventual consistency guarantees that if no new updates, all replicas converge to same value — but reads may return stale data temporarily. Acceptable for: social media likes/follower counts, CDN cached content, analytics dashboards, search index updates, shopping cart (with merge strategy). Not acceptable for: financial balances, inventory during checkout (without compensation), authorization decisions. Design with conflict resolution: last-write-wins, vector clocks, CRDTs for specific data types. Always communicate consistency guarantees to product team.

Q15: Compare SQL vs NoSQL – when to use each?

Answer: SQL (PostgreSQL, MySQL): structured data, complex joins, ACID transactions, ad-hoc queries, strong consistency — default choice for most applications. NoSQL document (MongoDB): flexible schema, nested documents, horizontal scaling, rapid iteration — content management, catalogs. Key-value (Redis): caching, sessions, rate limiting — not primary database. Wide-column (Cassandra): high write throughput, time-series, global distribution. Graph (Neo4j): relationship-heavy queries — social networks, fraud detection. Polyglot persistence: use multiple databases for different access patterns — not one-size-fits-all.

6. System Design HLD

TIER 4 — Study Month 6 (Days 141–154) • Prerequisites: [10b](#) → [10c](#) → [10-system-design-complete.pdf](#)

Q1: How do you approach a system design interview?

Answer: Follow structured framework: (1) Clarify requirements — functional (features) and non-functional (scale, latency, availability) for 5 minutes; (2) Back-of-envelope calculations — QPS, storage, bandwidth; (3) High-level design — boxes and arrows, main components; (4) Deep dive — 2-3 critical components interviewer chooses; (5) Bottlenecks and trade-offs. Communicate continuously — don't go silent for 10 minutes. Draw as you talk. Scope control: "For this interview, I'll focus on X and mention Y briefly." Interviewers evaluate process, not perfect architecture.

Q2: Design a URL shortener (TinyURL). Walk through key decisions.

Answer: Requirements: shorten URL, redirect, custom aliases, analytics. Scale: 100M URLs, 1000:1 read:write ratio. Design: API servers behind load balancer → application servers → database (PostgreSQL for URL mappings) + Redis cache for hot URLs. ID generation: base62 encoding of auto-increment ID or hash-based (MD5 first 7 chars with collision check). Redirect: 301 (permanent, cached by browser) vs 302 (track analytics). Cache: LRU cache + Redis — 80%

reads from cache. Analytics: async write to Kafka → processing pipeline. Storage estimate: $100M \times 500 \text{ bytes} \approx 50GB$ — single DB sufficient initially.

Q3: Explain load balancing strategies and algorithms.

Answer: Load balancer distributes traffic across servers — hardware (F5) or software (NGINX, HAProxy, AWS ALB). Algorithms: Round Robin (equal distribution, ignores load), Weighted Round Robin (more to powerful servers), Least Connections (route to least busy — good for varying request times), IP Hash (session stickiness), Consistent Hashing (minimal redistribution on node add/remove — caching, sharding). Layer 4 (TCP) vs Layer 7 (HTTP — path-based routing, SSL termination). Health checks remove unhealthy nodes. For global: GeoDNS + anycast.

Q4: What is caching and what are common caching patterns?

Answer: Cache stores frequently accessed data in fast storage (memory) to reduce latency and database load. Patterns: Cache-aside (app manages — check cache, on miss load DB, populate cache), Read-through (cache loads from DB transparently), Write-through (write to cache and DB together), Write-behind (write to cache, async flush to DB). Invalidation: TTL expiry, event-driven purge on update, cache versioning. CDN caches static assets at edge. Redis for application cache — sub-millisecond reads. Watch for: cache stampede (lock/mutex on miss), hot keys (local cache + replication).

Q5: How do you design for high availability (99.99%)?

Answer: $99.99\% = 52 \text{ minutes downtime/year}$. Strategies: eliminate single points of failure (multi-AZ deployment), redundancy at every layer (load balancers, app servers, databases with failover), health checks with automatic failover, circuit breakers prevent cascade failures, graceful degradation (disable non-critical features), multi-region for disaster recovery. Database: primary-replica with automatic failover (RDS Multi-AZ). Monitoring + alerting + on-call rotation. Chaos engineering validates failover. Trade-off: cost increases with redundancy — justify based on business impact of downtime.

Q6: Explain microservices vs monolith trade-offs.

Answer: Monolith: single deployable, simple debugging, ACID transactions easy, lower ops overhead — suits early stage and small teams. Microservices: independent deploy/scaling, team autonomy, technology diversity — adds network latency, distributed tracing needs, eventual consistency, deployment complexity. When to split: team size > 10-15 on same codebase, different scaling needs per component, clear bounded contexts. Strangler fig pattern: extract services incrementally. Many 80 LPA companies prefer "modular monolith" or "monolith first" — don't microservice prematurely.

Q7: What is a CDN and how does it work?

Answer: Content Delivery Network distributes cached content to edge servers geographically close to users — reduces latency and origin load. Flow: user request → DNS resolves to nearest edge → cache hit returns immediately; miss fetches from origin, caches, returns. Cache control via HTTP headers: `Cache-Control: max-age=3600`, `ETag` for validation. Use for: static assets (JS, CSS, images), video streaming, cacheable API responses. Providers: CloudFront, Cloudflare, Fastly. Invalidation: purge API on deploy. Dynamic content: CDN less useful unless edge computing (Cloudflare Workers).

Q8: Design a rate limiter for a distributed system.

Answer: Requirements: limit N requests per user/IP per window across all servers. Algorithms: token bucket (smooth rate, allows bursts), sliding window log (precise, memory-heavy), sliding window counter (approximate, efficient). Distributed implementation: Redis with atomic operations — `INCR` with TTL, or sorted set for sliding window log. Return 429 with `Retry-After` header. Consider: different limits per tier (free vs paid), whitelist internal services, rate limit at API gateway level. Fail-open vs fail-closed on Redis outage — product decision.

Q9: Explain message queues and when to use them.

Answer: Message queues decouple producers and consumers — async processing, load leveling, reliability. Use cases: email sending, image processing, order fulfillment, event sourcing, inter-service communication. Kafka: high throughput, log-based, replay capability — event streaming. RabbitMQ/SQS: traditional queue, message deleted after consumption. Patterns: pub/sub (fan-out), point-to-point (competing consumers), request-reply. Guarantees: at-most-once, at-least-once (with idempotent consumers), exactly-once (complex — Kafka transactions). Always design consumers to be idempotent.

Q10: How do you estimate system capacity (back-of-envelope)?

Answer: Framework: (1) Estimate DAU → peak concurrent users (10% of DAU) → QPS (peak users × requests/user/86400 × peak factor 3-5x); (2) Storage: users × data per user × growth factor × retention period; (3) Bandwidth: QPS × average response size; (4) Memory: working set for cache (20% of hot data). Example: 10M DAU, 10 requests/day average → $10M \times 10 / 86400 \approx 1200$ QPS average, ~6000 QPS peak. Each server handles ~1000 QPS → 6-10 app servers. Practice mental math — interviewers care about approach, not exact numbers.

Q11: What is consistent hashing and why is it used?

Answer: Consistent hashing maps keys and servers to a hash ring — adding/removing a server only remaps K/n keys (K =total keys, n =nodes) vs full remapping with modulo hashing. Virtual nodes improve load distribution. Used for: distributed caches (Memcached), sharding, load balancers, CDNs. When server added: only adjacent key ranges move. Implementation: hash key to ring position, walk clockwise to find responsible server. Handles heterogeneous server capacities via virtual node weighting.

Q12: Design a notification system (push, email, SMS).

Answer: Components: event source → notification service → channel routers (push/email/SMS) → third-party providers (FCM, SendGrid, Twilio). Queue between service

and providers for reliability and rate limiting. User preferences: channel priority, quiet hours, opt-out per channel. Template engine for content personalization. Idempotency keys prevent duplicate notifications. Retry with exponential backoff for transient failures. Dead letter queue for failed notifications. Scale: millions of notifications/day — partition queue by channel, batch email sends. Track delivery status asynchronously.

Q13: Explain database scaling strategies.

Answer: Vertical scaling: bigger machine — simple, limited ceiling, downtime for upgrade. Read replicas: scale reads, replication lag concerns. Caching: Redis for hot data — 80/20 rule. Database sharding: split data across instances — complex cross-shard queries. Federation: split by feature (users DB, products DB). Denormalization: reduce joins at cost of consistency. CQRS: separate read and write models — optimize each independently. Archiving: move old data to cold storage. Choose based on bottleneck: CPU → vertical/read replicas; storage → sharding; read latency → cache.

Q14: What are API gateways and their responsibilities?

Answer: API gateway is single entry point for client requests — routes to appropriate microservices. Responsibilities: authentication/authorization, rate limiting, SSL termination, request routing, load balancing, response aggregation (BFF pattern), protocol translation (REST to gRPC), caching, logging/monitoring. Examples: Kong, AWS API Gateway, NGINX. Prevents clients from knowing internal service topology. Trade-off: potential bottleneck and single point of failure — deploy with redundancy. GraphQL gateway aggregates multiple REST services.

Q15: Design a chat/messaging system like WhatsApp.

Answer: Components: clients → WebSocket gateway → message service → message store (Cassandra for write-heavy) + Redis (online status, recent messages). Flow: User A sends message → gateway → message service → store → push to User B via WebSocket (if online) or FCM/APNs (if offline). Group chat: fan-out on write (small groups) vs fan-out on read (large groups). Delivery receipts: sent → delivered → read — separate status events. End-to-end

encryption: client-side key exchange. Scale: billions of messages/day — partition by conversation ID, message queue for async delivery.

Q16: Explain the circuit breaker pattern.

Answer: Circuit breaker prevents cascade failures when downstream service is unhealthy. States: Closed (normal, requests pass), Open (failures exceed threshold — fail fast without calling downstream), Half-Open (periodic test request — if success, close; if fail, reopen). Benefits: fail fast reduces latency wasted on timeouts, gives downstream time to recover. Implementation: libraries (resilience4j, opossum) or service mesh (Istio). Configure: failure threshold, timeout duration, half-open retry interval. Combine with fallback responses for graceful degradation.

Q17: What is event-driven architecture?

Answer: Services communicate via events (state changes) rather than direct calls — loose coupling, async processing, audit trail. Components: event producers, event broker (Kafka), event consumers. Patterns: event notification (fire-and-forget), event-carried state transfer (event contains data), event sourcing (events as source of truth). Benefits: scalability, resilience, easy to add new consumers. Challenges: eventual consistency, debugging distributed flows, schema evolution. CQRS often paired with event sourcing for complex domains.

Q18: Design a video streaming platform like YouTube.

Answer: Upload: client → upload service → blob storage (S3) → transcoding pipeline (multiple resolutions via worker queue) → CDN distribution. Playback: client requests video → CDN serves segments (HLS/DASH adaptive bitrate) → player switches quality based on bandwidth. Metadata: PostgreSQL for video info, Elasticsearch for search. Scale: popular videos from CDN cache (80%+ cache hit), long-tail from origin. Thumbnails, comments as separate services. Recommendation engine: offline batch processing + real-time features. Storage: 1 hour 1080p ≈ 3GB — petabytes total.

Q19: How do you handle idempotency in distributed systems?

Answer: Idempotent operation produces same result regardless of how many times executed — critical for retry safety. Implementation: idempotency key (client-generated UUID) stored with result — duplicate requests return cached result. HTTP: PUT and DELETE are naturally idempotent; POST needs idempotency key. Payment: check if transaction ID already processed before charging. TTL on idempotency records (24-48 hours). Database: unique constraint on idempotency key. Message queues: consumer checks processed message IDs before handling.

Q20: Explain multi-region deployment and data sovereignty.

Answer: Multi-region: deploy in multiple geographic regions for low latency and disaster recovery. Active-active (all regions serve traffic) vs active-passive (standby for DR). Challenges: data replication across regions (latency, consistency), conflict resolution for concurrent writes, increased cost. Data sovereignty: regulations require data stored in specific countries (GDPR, India data localization). Solutions: region-specific databases, geo-routing (Route 53 latency-based routing), CRDTs for conflict-free replication. CAP theorem bites harder across regions — design for eventual consistency globally.

7. LLD

TIER 3 — Study Month 5 (Days 113–140) • Prerequisites: [09-lll-complete.pdf](#) , [15-machine-coding-complete.pdf](#)

Q1: Explain all five SOLID principles with examples.

Answer: Single Responsibility: one class, one reason to change — `InvoiceGenerator` separate from `EmailSender`. Open/Closed: extend via new classes, not modifying existing — add new payment method via `PaymentStrategy` interface. Liskov Substitution: subclasses usable wherever parent is — `Square` must not break `Rectangle` `setWidth/setHeight` expectations. Interface Segregation: small focused interfaces — `Readable` / `Writable` instead of fat `FileOps`. Dependency Inversion: depend on abstractions — `OrderService` depends on `PaymentGateway`

interface, not Stripe class. SOLID reduces coupling and increases testability — mention specific principle when designing in interviews.

Q2: When would you use Strategy vs Factory vs Observer pattern?

Answer: Strategy: interchangeable algorithms at runtime — payment methods, pricing rules, sorting. Factory: object creation logic centralized — create different notification senders based on type. Observer: one-to-many dependency — event system, UI model-view updates. Strategy vs Factory: Strategy switches behavior of existing object; Factory creates different objects. Observer vs Pub/Sub: Observer is direct; Pub/Sub has intermediary broker (more decoupled). Don't force patterns — use when variation point exists in requirements.

Q3: Design a parking lot system – key classes and relationships.

Answer: Core entities: `ParkingLot` (manages floors), `Floor` (contains spots), `ParkingSpot` (type, availability), `Vehicle` (type enum), `Ticket` (entry time, spot reference). Services: `SpotAllocator` (finds compatible spot), `FeeCalculator` (strategy per vehicle type). Patterns: Strategy for fee calculation, Factory for spot creation. Key design decision: spot types (small/medium/large) mapped to vehicle types. Extensibility: add `ElectricSpot` without changing allocator by extending spot type enum and strategy.

Q4: Explain Singleton pattern – why is it often considered an anti-pattern?

Answer: Singleton ensures one instance globally — useful for config managers, connection pools, loggers. Problems: hidden dependencies (hard to test — can't mock), global state, violation of SRP, thread safety complexity. In Node.js (single-threaded), less concern about thread safety but testing remains hard. Alternatives: dependency injection (pass instance), module scope (Node modules are naturally singleton), container-managed lifecycle. Interview answer: know the pattern, acknowledge drawbacks, prefer DI in modern code.

Q5: What is the difference between Factory and Abstract Factory?

Answer: Factory Method: single method creates one type of object —

`createNotification(type)` returns Email/SMS/Push. Abstract Factory: factory of factories — creates families of related objects — `UIFactory` creates Button + Checkbox + Dialog for Windows vs Mac theme. Use Factory Method when creation logic is simple and single-product. Use Abstract Factory when system must work with multiple product families consistently. Both hide instantiation details from client code.

Q6: Design an LRU Cache – data structures and complexity.

Answer: HashMap + Doubly Linked List achieves $O(1)$ get and put. HashMap maps key → node for $O(1)$ lookup. Doubly linked list maintains access order — most recent at head, least recent at tail. Get: lookup node, move to head. Put: if exists update and move to head; if new and at capacity, remove tail node and add new at head. Space: $O(\text{capacity})$. Alternative: JavaScript Map maintains insertion order but get doesn't reorder — need manual list for true LRU.

Q7: Explain Command pattern with a real example.

Answer: Command encapsulates request as object — enables undo, redo, queuing, logging. Example: text editor undo — each edit is a Command with `execute()` and `undo()`. Remote control: button press creates command, receiver executes. Structure: Command interface, ConcreteCommands, Invoker (calls execute), Receiver (does work). In React: actions in Redux are commands. Transaction systems: each operation is command that can be rolled back. Useful when operations need to be deferred, logged, or reversed.

Q8: What is the Adapter pattern? When do you need it?

Answer: Adapter converts interface of a class into another interface clients expect — wraps incompatible API. Example: legacy payment API uses XML, new system expects JSON — adapter translates. Third-party library with different method names — adapter maps to your

interface. Two types: object adapter (composition — preferred) and class adapter (inheritance). Common in integration work — wrapping external SDKs behind your domain interface for testability and future vendor swap.

Q9: Design a vending machine using State pattern.

Answer: States: Idle, HasMoney, Dispensing, OutOfStock. Each state class implements `insertMoney()`, `selectProduct()`, `dispense()`, `cancel()`. VendingMachine holds current state and delegates actions. Transitions: Idle + money → HasMoney; HasMoney + select → Dispensing; Dispensing → complete → Idle. Benefits: no complex if/else chains, each state's behavior isolated, easy to add states (MaintenanceMode). State pattern vs Strategy: State manages internal transitions; Strategy is chosen externally.

Q10: Explain dependency injection and its benefits.

Answer: Dependency Injection provides dependencies from outside rather than creating internally — constructor injection (preferred), setter injection, interface injection. Benefits: testability (inject mocks), loose coupling (depend on interfaces), configuration flexibility (swap implementations). Manual DI: pass dependencies in constructor. Framework DI: tsyringe, InversifyJS, NestJS built-in. Anti-pattern: service locator (hidden dependencies). In interviews, show constructor injection with interface — `class OrderService { constructor(private payment: PaymentGateway) {} }`.

Q11: What is the difference between aggregation and composition?

Answer: Both are "has-a" relationships. Composition: strong ownership — child cannot exist without parent (House has Rooms — room destroyed with house). Aggregation: weak ownership — child can exist independently (Department has Employees — employee exists without department). In code: composition = create child in constructor, don't expose for external deletion. Aggregation = reference externally created objects. UML: composition filled diamond, aggregation hollow diamond. Affects lifecycle management and design decisions.

Q12: Design Splitwise – how do you track who owes whom?

Answer: On expense add: for each participant except payer, increment debt[participant][payer] by their share. Use nested Map or balance sheet class. Split strategies: Equal (total/participants), Exact (specified amounts), Percentage (proportional). Simplification: compute net balance per person (creditors positive, debtors negative), greedy match largest creditor with largest debtor. Extensibility: groups contain users, expenses belong to groups, balances computed per group. Graph representation: nodes are users, edges are debts with amounts.

Q13: Explain Template Method pattern.

Answer: Template Method defines algorithm skeleton in base class — subclasses override specific steps without changing structure. Example: data export pipeline — base class defines `export()`: `fetchData` → `transform` → `write`. `CSVExport` overrides `transform/write`; `JSONExport` overrides `write`. Hollywood Principle: "don't call us, we'll call you" — base class controls flow. vs Strategy: Template Method uses inheritance for variation; Strategy uses composition. Use when algorithm structure is fixed but steps vary.

Q14: What is the Builder pattern and when to use it?

Answer: Builder constructs complex objects step by step — separates construction from representation. Use when: many optional parameters (avoid telescoping constructor), complex object with multiple representations, step-by-step construction required. Example: `new QueryBuilder().select('*').from('users').where('active', true).limit(10).build()`. Fluent interface returns `this` for chaining. vs Factory: Factory creates in one step; Builder assembles over multiple calls. JavaScript/TypeScript: often replaced by object literals with defaults or options pattern for simpler cases.

Q15: How do you make LLD designs extensible for future requirements?

Answer: Apply Open/Closed Principle — identify variation points and extract interfaces. Use enums + strategy for types that will grow (vehicle types, payment methods). Factory for object creation when types expand. Avoid hard-coded conditionals on type strings. Composition over inheritance for flexibility. Document extension points explicitly: "To add new X, implement Y interface and register in Z." Mention specific future requirements and show how design accommodates them without modification — interviewers often ask "what if we add...?"

8. Distributed Systems

TIER 4 — Study Month 6 (Days 151–158) • Prerequisites: [11-distributed-systems-complete.pdf](#) (read theory BEFORE these Q&A)

Q1: What is the Two Generals Problem and its implication?

Answer: Two generals must agree on attack time via unreliable messengers — impossible to guarantee consensus with message loss. Implication: perfect consensus is impossible in asynchronous unreliable networks. Leads to practical solutions: accept partial failure, use timeouts, design for eventual consistency. TCP doesn't solve this — it guarantees delivery between two points but not end-to-end business confirmation. Informs why distributed transactions use 2PC/3PC with trade-offs or avoid distributed transactions entirely via sagas.

Q2: Explain the Saga pattern for distributed transactions.

Answer: Saga manages distributed transaction as sequence of local transactions with compensating actions for rollback. Choreography: each service publishes events, others react — no coordinator, harder to track. Orchestration: central coordinator directs steps and compensations — clearer flow, single point of control. Example: order saga — reserve inventory → charge payment → create shipment. If payment fails → compensate inventory release. Compensating transactions aren't always perfect (can't un-send email). Prefer idempotent steps. Alternative to 2PC which blocks and doesn't scale.

Q3: What is a distributed lock and how do you implement one?

Answer: Distributed lock ensures only one process accesses a resource across cluster. Redis implementation: `SET key value NX EX 30` (set if not exists with expiry). Must include unique value to safely release (check value before delete — Lua script for atomicity). Redlock algorithm: acquire on N Redis instances for fault tolerance. ZooKeeper/etcd: ephemeral sequential nodes — naturally handles process crash (node deleted on disconnect). Always set TTL to prevent deadlock on crash. Fencing tokens prevent delayed old lock holder from writing.

Q4: Explain leader election and why it's needed.

Answer: Leader election selects one node as coordinator — needed for: single writer scenarios, scheduled job execution, shard assignment. Algorithms: Raft (understandable consensus), Paxos (theoretical foundation), Bully (highest ID wins — simple but noisy). etcd and ZooKeeper implement Raft/ZAB for election. Kubernetes uses etcd for leader election via lease-based locks. On leader failure, new election within seconds. Design services to handle leader change gracefully — new leader takes over work.

Q5: What is gossip protocol and where is it used?

Answer: Gossip protocol spreads information peer-to-peer like epidemic — each node periodically shares state with random peers. Eventually all nodes converge. Used in: Cassandra cluster membership, DynamoDB, Consul service discovery, failure detection. Properties: eventually consistent, highly scalable, fault tolerant, no single point of failure. Trade-offs: slow convergence, bandwidth usage, complexity in debugging. Anti-entropy: periodic full state comparison to fix drift. Gossip suits cluster state, not critical real-time data.

Q6: Explain vector clocks and conflict resolution.

Answer: Vector clock tracks causality across distributed nodes — each node maintains vector of counters per node. On event: increment own counter. On send: attach vector. On receive: merge

(max per element). Compare: if all elements $\geq$ and at least one $>$, happened-before; if neither $\geq$ other, concurrent (conflict). Used in: Dynamo-style databases, Riak, CRDTs. Conflict resolution on concurrent writes: last-write-wins (loses data), application merge, CRDT automatic merge. Essential for understanding eventual consistency semantics.

Q7: What is the difference between at-least-once, at-most-once, and exactly-once delivery?

Answer: At-most-once: fire and forget — may lose messages, never duplicates. Fast, used for metrics/logging. At-least-once: retry until acknowledged — may duplicate, never lose. Requires idempotent consumers. Exactly-once: each message processed once — hardest, achieved via idempotent producers + transactional consumers + deduplication (Kafka transactions, unique message IDs). Most systems implement at-least-once with idempotent processing — simpler and sufficient. Exactly-once end-to-end requires cooperation across producer, broker, and consumer.

Q8: Explain service discovery – client-side vs server-side.

Answer: Service discovery locates service instances dynamically — essential in microservices where instances scale and change. Client-side (Eureka): client queries registry, caches instances, load balances locally — Netflix model. Server-side (Consul, K8s kube-dns): load balancer/proxy queries registry, client talks to LB — simpler client. Kubernetes: Services + DNS — `my-service.namespace.svc.cluster.local`. Health checks remove unhealthy instances. Service mesh (Istio) adds sidecar proxy handling discovery, retries, circuit breaking transparently.

Q9: What is a bloom filter and its use cases?

Answer: Bloom filter is probabilistic data structure testing set membership — may have false positives (says "maybe present") but never false negatives ("definitely not present"). Space efficient: ~ 10 bits per element for 1% false positive rate. Cannot delete (unless counting bloom filter). Use cases: database query optimization (avoid disk reads for non-existent keys), CDN

cache, distributed cache (check before network call), web crawler URL dedup. In Cassandra: check bloom filter before SSTable scan. Trade-off: tune false positive rate vs memory.

Q10: Explain the thundering herd problem and solutions.

Answer: Thundering herd: many clients simultaneously request same resource when it becomes available or expires — overwhelms server/cache/database. Scenarios: cache expiry (1000 requests hit DB simultaneously), lock release, server restart. Solutions: request coalescing (single request fetches, others wait), jitter on TTL (randomize expiry), mutex per cache key (only one thread fetches on miss), circuit breaker, pre-warming cache before expiry. In distributed systems: distributed lock on cache miss handler ensures single fetcher.

Q11: What is eventual consistency vs strong consistency?

Answer: Strong consistency: after write completes, all reads return new value — linearizable. Eventual consistency: reads may return stale value temporarily, but converge if no new writes. Strong: PostgreSQL single node, ZooKeeper, etcd. Eventual: Cassandra, DynamoDB (default), DNS, CDN. Causal consistency: middle ground — causally related operations seen in order. For 80 LPA: discuss choosing based on business requirement — payment balance needs strong; social media like count can be eventual. PACELC theorem extends CAP for normal operation.

Q12: Explain horizontal vs vertical partitioning (sharding).

Answer: Vertical partitioning: split columns — hot columns in one table, cold in another. Reduces row size, improves cache efficiency. Horizontal partitioning (sharding): split rows across instances by shard key — `user_id` hash mod N. Shard key must match query patterns — bad key causes scatter-gather across all shards. Resharding: consistent hashing minimizes data movement. Cross-shard queries expensive — design to avoid (denormalize, query aggregator). MongoDB, Cassandra shard natively; PostgreSQL via Citus or application-level.

Q13: What is a heartbeat mechanism in distributed systems?

Answer: Heartbeat: periodic signal proving node is alive — missed heartbeats trigger failover. Implementation: node sends heartbeat every N seconds to coordinator/peers; if 3 consecutive missed, mark failed. Used in: leader election, load balancer health checks, cluster membership (Cassandra gossip), Kubernetes liveness probes. False positives: GC pause causes missed heartbeat — use phi accrual failure detector (Cassandra) for adaptive thresholds. Split-brain: two nodes both think they're leader — use quorum/fencing to prevent.

Q14: Explain MapReduce and its modern alternatives.

Answer: MapReduce: programming model for large-scale data processing — Map (transform/filter) → Shuffle (group by key) → Reduce (aggregate). Hadoop popularized it. Limitations: batch only, high latency, complex for iterative algorithms. Modern alternatives: Spark (in-memory, faster iterative), Flink (stream processing), Kafka Streams (real-time on Kafka). Cloud: BigQuery, Snowflake for SQL-based analytics. MapReduce concept still relevant — understand pattern even if you use Spark. For real-time: stream processing replaces batch for many use cases.

Q15: What is a service mesh and when do you need one?

Answer: Service mesh (Istio, Linkerd) provides infrastructure layer for service-to-service communication via sidecar proxies. Features: mTLS, traffic management (canary, retry, timeout), observability (metrics, tracing), circuit breaking — without modifying application code. Need when: 20+ microservices, security compliance requires mTLS, complex traffic routing, unified observability. Don't need when: monolith or few services — NGINX/API gateway sufficient. Cost: sidecar resource overhead, operational complexity. Trend: eBPF-based meshes reduce sidecar overhead.

9. Behavioral / Senior

TIER 4 — Study Month 6 (Days 141–168) • Prerequisites: [16-behavioral-senior-complete.pdf](#)

Q1: Tell me about a time you disagreed with your manager.

Answer: Use STAR format. Situation: technical disagreement on approach. Action: data-driven discussion, documented trade-offs, proposed experiment/spike. Result: outcome + relationship preserved. Key: show respect for hierarchy while advocating with evidence. Never say "I was right and they were wrong" — say "we aligned on the approach best supported by data." Demonstrates: communication, humility, influence without authority. See [16-behavioral-senior-complete.md](#) for full story templates.

Q2: Describe your most significant technical impact.

Answer: Structure: Problem (business impact) → Your role (specific) → Actions (technical decisions) → Results (metrics). Example: "Improved payment success rate by 4.2% through retry engine redesign — 15 crore GMV recovered annually." Must include numbers. Scope beyond your team shows L5 readiness. Mention: what you learned, what you'd do differently. Avoid team-only credit — clarify your contribution explicitly.

Q3: How do you handle technical debt vs feature pressure?

Answer: Framework: categorize debt (critical — affects reliability/security vs cosmetic), quantify impact (incident frequency, developer velocity loss), propose allocation (20% sprint capacity for debt — industry standard). Communicate to PM in business terms: "This debt causes 2 incidents/month costing 4 engineer-hours each." Never say "PMs don't understand tech debt" — translate to business impact. Prioritize: security/reliability debt before code style. Document debt in backlog with ROI estimate for payoff.

Q4: Tell me about a time you mentored someone successfully.

Answer: STAR with specific growth metrics: "Junior engineer went from $3\times$ average ticket time to team average in 75 days." Actions: structured plan, pairing (not doing their work), teaching frameworks not answers, advocating in calibration. Result: promotion, they now mentor others. Shows: people development — critical for L5+. Different from "I answered their questions" — show systematic approach to growth.

Q5: How do you make decisions with incomplete information?

Answer: Framework: (1) Identify what you know vs need to know; (2) Assess cost of waiting vs cost of wrong decision (reversible vs irreversible — Bezos Type 1/2); (3) Gather minimum viable data — spike, consult experts, check analogous systems; (4) Make decision with explicit assumptions documented; (5) Set review checkpoint to validate. Example: "Chose Redis over Memcached based on persistence need — documented assumption, validated after 1 month with hit rate metrics." Shows judgment, not recklessness.

Q6: Tell me about a failure and what you learned.

Answer: Choose real failure where you take ownership — not "we failed" but "I failed because I..." Structure: what happened, your role, immediate recovery, systemic fix, lesson applied since. Example: production outage from your deploy → rollback → postmortem → new deployment policy. Never blame others. Show: accountability, learning orientation, systemic thinking (not just fixing the bug, preventing class of bugs). Interviewers at 80 LPA specifically probe for ego and blame-shifting.

Q7: Why do you want to join this company at this level?

Answer: Specific, researched answer — not generic "great culture." Mention: specific team/product, technical challenges aligned with your experience, scope increase from current role, engineers you admire (LinkedIn/blog posts). Connect your background to their problems: "I've solved payment reliability at scale; your UPI volume is the next level challenge." Avoid:

money as primary reason, negative comments about current employer, "I want FAANG on resume."

Q8: How do you prioritize when everything is P0?

Answer: Framework: Impact × Urgency matrix shared visibly with stakeholders. Force-rank with requestors in same room — not via Slack wars. Criteria: legal/compliance deadlines (non-negotiable), revenue impact, user count affected, reversibility. Example: "Security CVE scored 9.8 → fixed in 48h; feature delayed 1 sprint with PM agreement and MVP scope reduction." Shows: stakeholder management, business judgment, transparency. Never: "I just work harder" or "I do whatever PM says."

Q9: Describe a time you improved team productivity.

Answer: Quantify before/after: "PR review time from 24h to 4h median." Systemic fix, not heroics: CODEOWNERS, review SLAs, PR templates, paired review training. Show: process thinking, measurement, sustainable improvement. Different from "I worked faster" — improved the system, not just personal output. L5 signal: impact on team/org, not individual contribution alone.

Q10: How do you stay current with technology?

Answer: Balanced approach: curated sources (specific newsletters, conference talks — React Conf, Strange Loop), hands-on (side projects, contributing to OSS), learning from production (postmortems, architecture reviews), internal tech talks. Avoid: "I read Hacker News daily" (passive). Show: depth over breadth — "I deep-dived into CRDTs after our collaboration feature had conflict bugs." Senior engineers evaluate new tech critically, not adopt hype-driven.

Q11: Tell me about a time you had to say no to a stakeholder.

Answer: STAR: Situation where unreasonable request (timeline, scope, technical feasibility). Action: explain why with data, propose alternative, escalate with options not problems. Result: aligned solution, relationship maintained. Key: "say no to the request, not the person." Example: "PM wanted 3-week feature; showed 6-week estimate with risk doc; agreed on MVP in 3 weeks." Shows: backbone, communication, business partnership.

Q12: How do you approach code reviews?

Answer: Principles: review for correctness, design, readability, tests — not style (automate with linter). Ask questions rather than dictate: "Have you considered what happens if this returns null?" Turnaround: same day for small PRs, 24h max. Praise good patterns publicly. For senior reviews: focus on architecture, extensibility, operational concerns (monitoring, rollback). Mentoring through reviews: explain why, link to docs, don't just request changes. Balance: thoroughness vs blocking velocity.

Q13: Describe leading a project across multiple teams.

Answer: STAR emphasizing: alignment mechanisms (RFC, weekly sync, decision log), proactive communication, unblocking others, not doing all work yourself. Metrics: delivered on time despite cross-team dependencies. Tools: shared doc, clear DRI (Directly Responsible Individual) per workstream, escalation path defined upfront. L5 signal: influence without authority, organizational navigation. Mention specific conflict resolution if it arose.

Q14: What would your team say about you in a 360 review?

Answer: Honest self-awareness — one strength with evidence, one growth area with improvement actions. Example: "They'd say I'm strong technically and helpful in debugging, but I've been working on listening more in design discussions instead of jumping to solutions — I now write my thoughts in doc comments before meetings." Avoid: purely positive (seems unaware) or purely negative. Shows: self-awareness, growth mindset — key L5 trait.

Q15: Where do you see yourself in 3-5 years?

Answer: Focus on scope and impact, not titles: "Driving technical strategy for a product area, mentoring senior engineers, representing engineering in product decisions." Show flexibility: "Staff IC or eng manager depending on what I discover I enjoy — this role helps me explore that." Connect to company: "Google's L5 scope is exactly the cross-team impact I'm seeking." Avoid: "Your CEO's job" (unrealistic) or "I don't know" (unfocused).

10. Salary / Negotiation

TIER 5 — Study when offer received • Prerequisites: completed interview loop

Q1: How should you research compensation for 80 LPA roles?

Answer: Use multiple sources: Levels.fyi (most accurate for FAANG — filter by level, location, company), AmbitionBox/Glassdoor (Indian companies — verify with multiple data points), Blind (anonymous but verify), network conversations with people at target companies. Understand components: base, variable/bonus, RSU/stock (4-year vest, 1-year cliff), signing bonus, ESOP (startups). 80 LPA in India might be 40 base + 20 variable + 20 RSU annual value. Always compare total compensation (TC), not just base. Factor: cost of living, tax differences between cities.

Q2: When and how should you disclose current compensation?

Answer: Indian companies often ask current CTC — you can redirect: "I'm focused on the role's scope and market value for this level. Based on my research, the range for L5 at this company is X-Y, and I'm targeting the upper half given my experience in [specific area]." Never lie about current comp — background checks can verify. If forced: give total comp including all components, not just base. New trend: some states/countries ban salary history questions — know your rights. Best: let them make first offer when possible.

Q3: How do you negotiate an offer without losing it?

Answer: Principles: always negotiate (companies expect it, initial offer has room), be collaborative not adversarial ("I'm excited about this role, and I'd like to discuss the compensation"), provide justification (competing offer, market data, specific skills), negotiate multiple components (if base is fixed, ask for signing bonus, extra RSU, relocation). Never: ultimatum unless you mean it, negotiate before receiving written offer, compare offers dishonestly. Timeline: "I need until [date] to decide" — usually 1 week. Get final offer in writing.

Q4: How do you compare offers from FAANG vs Indian unicorn vs startup?

Answer: Framework beyond TC: (1) Level/scope — Google L5 vs startup "Staff" may differ vastly; (2) Learning — technology, scale, mentorship; (3) Growth — promotion timeline, performance review culture; (4) Risk — startup ESOP may be worth \$0 or 10×; (5) Work-life — verify with Blind/connections, not recruiter; (6) Brand — resume value for next switch. Decision matrix: weight factors by your priorities. 80 LPA startup ESOP: ask percentage, latest valuation, liquidation preference, vesting schedule. FAANG RSU: liquid, predictable. No universally correct choice.

Q5: What are common compensation mistakes at 80 LPA level?

Answer: Mistakes: (1) Optimizing only for base, ignoring RSU/ESOP value; (2) Not negotiating signing bonus (easiest to increase); (3) Accepting level downgrade without TC adjustment; (4) Ignoring vesting schedule — joining near stock refresh vs just after; (5) Not asking about variable pay structure (guaranteed vs performance); (6) Comparing India TC to US TC without PPP adjustment; (7) Not getting competing offer for leverage; (8) Revealing excitement before negotiating ("I accept!" before discussing numbers). Best practice: negotiate after all rounds complete, with written offer, using market data.

Quick Reference – Question Count by Category

Category	Questions
JavaScript Deep	15
TypeScript	10
React / Frontend	15
Node.js / Backend	15
Databases	15
System Design HLD	20
LLD	15
Distributed Systems	15
Behavioral / Senior	15
Salary / Negotiation	5
Total	140

Target: Google L5, Meta E5, Amazon L6, Staff Engineer at Indian unicorns — 80+ LPA comprehensive interview preparation.